

Assignment 1: Resource Governance and Failure Modes

Objective

Validate practical understanding of Requests, Limits, LimitRange, ResourceQuota, OOMKilled behavior, and quota enforcement.

Resources Created

- Namespace: intern-test
- LimitRange: intern-limitrange
- ResourceQuota: intern-quota
- Pod: good-pod
- Pod: oom-pod

Validation

LimitRange

Configured default CPU and memory requests/limits for containers.

ResourceQuota

Configured hard limits for:

- Pods
- requests.cpu
- requests.memory
- limits.cpu
- limits.memory

Successful Pod

good-pod was created successfully and inherited resource values from LimitRange.

OOMKilled Pod

oom-pod was configured with a low memory limit and intentionally exceeded it.

Result:

- Pod entered CrashLoopBackOff state
- Container termination reason: OOMKilled

Screenshots:

LimitRange:

```
NAME                STATUS    AGE
apache              Active    25h
default             Active    16d
dev-node            Active    45m
intern-test         Active    17s
kube-node-lease     Active    16d
kube-public         Active    16d
kube-system         Active    16d
mysql               Active    15d
notes-app           Active    15d
ubuntu@ip-172-31-34-75:~/Radware-Assignment$ mkdir assignment1 assignment2 assignment3 assignment4 assignment5
ubuntu@ip-172-31-34-75:~/Radware-Assignment$ kubectl get nodes
NAME                STATUS    ROLES    AGE    VERSION
minikube            Ready     control-plane  16d    v1.35.1
ubuntu@ip-172-31-34-75:~/Radware-Assignment$ cd assignment1
ubuntu@ip-172-31-34-75:~/Radware-Assignment/assignment1$ LimitRange.yml
LimitRange.yml: command not found
ubuntu@ip-172-31-34-75:~/Radware-Assignment/assignment1$ vim LimitRange.yml
ubuntu@ip-172-31-34-75:~/Radware-Assignment/assignment1$ kubectl apply -f LimitRange.yml
limitrange/default-limits created
ubuntu@ip-172-31-34-75:~/Radware-Assignment/assignment1$ kubectl get limitrange -n intern-test
NAME                CREATED AT
default-limits      2026-06-16T07:54:17Z
ubuntu@ip-172-31-34-75:~/Radware-Assignment/assignment1$ kubectl describe limitrange default-limits -n intern-test
Name:         default-limits
Namespace:    intern-test
Type:         Resource
              Min    Max    Default Request  Default Limit  Max Limit/Request Ratio
-----
Container     cpu    -      200m             500m            -
Container     memory -      256Mi            512Mi            -
```

ResourceQuota:

```
ubuntu@ip-172-31-34-75: ~/R  X + v
🔥 Starting "minikube" primary control-plane node in "minikube" cluster
📦 Pulling base image v0.0.50 ...
🔄 Restarting existing docker container for "minikube" ...
🔧 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔍 Verifying Kubernetes components...
  ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Enabled addons: storage-provisioner, default-storageclass
🎉 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
ubuntu@ip-172-31-34-75:~$ ls
Radware-Assignment  kubernetes-tutorial
ubuntu@ip-172-31-34-75:~$ cd Radware--Assignment
-bash: cd: Radware--Assignment: No such file or directory
ubuntu@ip-172-31-34-75:~$ cd Radware-Assignment
ubuntu@ip-172-31-34-75:~/Radware-Assignment$ cd assignment1
ubuntu@ip-172-31-34-75:~/Radware-Assignment/assignment1$ ls
LimitRange.yml
ubuntu@ip-172-31-34-75:~/Radware-Assignment/assignment1$ vim ResourceQuota.yml

[1]+  Stopped                  vim ResourceQuota.yml
[1]+  Stopped                  vim ResourceQuota.yml
ubuntu@ip-172-31-34-75:~/Radware-Assignment/assignment1$ ls
LimitRange.yml
ubuntu@ip-172-31-34-75:~/Radware-Assignment/assignment1$ vim ResourceQuota.yml
ubuntu@ip-172-31-34-75:~/Radware-Assignment/assignment1$ kubectl apply -f ResourceQuota.yml
resourcequota/intern-quota created
ubuntu@ip-172-31-34-75:~/Radware-Assignment/assignment1$ kubectl get resourcequota intern-quota -n intern-test
NAME          REQUEST          LIMIT          AGE
intern-quota  pods: 0/3, requests.cpu: 0/1, requests.memory: 0/1Gi  limits.cpu: 0/2, limits.memory: 0/2Gi  2m
```

OOMKilled Container:

```
ubuntu@ip-172-31-34-75: ~/R x + v
--vm-hang
1
State:      Waiting
Reason:     CrashLoopBackOff
Last State: Terminated
Reason:     OOMKilled
Exit Code:  1
Started:    Tue, 16 Jun 2026 13:34:36 +0000
Finished:   Tue, 16 Jun 2026 13:34:36 +0000
Ready:      False
Restart Count: 1
Limits:
  cpu:      500m
  memory:   50Mi
Requests:
  cpu:      200m
  memory:   25Mi
Environment: <none>
Mounts:
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-vs4kz (ro)
Conditions:
  Type                               Status
PodReadyToStartContainers          True
Initialized                         True
Ready                              False
ContainersReady                    False
PodScheduled                        True
Volumes:
  kube-api-access-vs4kz:
    Type:  Projected (a volume that contains injected data from multiple sources)
```

AdmissionError:

```
ubuntu@ip-172-31-34-75: ~/R x + v
ubuntu@ip-172-31-34-75:~/Radware-Assignment/assignment1$ kubectl run pod3 --image=nginx -n intern-test
pod/pod3 created
ubuntu@ip-172-31-34-75:~/Radware-Assignment/assignment1$ kubectl run pod4 --image=nginx -n intern-test
Error from server (Forbidden): pods "pod4" is forbidden: exceeded quota: intern-quota, requested: pods=1, used: pods=3,
limited: pods=3
ubuntu@ip-172-31-34-75:~/Radware-Assignment/assignment1$ ~|
```

Quota Enforcement

After reaching the pod quota limit, additional pod creation was denied.

Example error:

"exceeded quota: intern-quota"

OOMKill vs Quota Denial

OOMKill occurs when a running container exceeds its memory limit. Kubernetes terminates the container to protect node stability. The pod may restart depending on its restart policy.

Quota denial occurs before a resource is created. The Kubernetes API server rejects the request because the namespace has exceeded configured resource limits.

OOMKill affects running workloads, while quota denial prevents new workloads from being created.